Here's a **real-time industry view** of the **types of deployment models possible**, with **when to use**, **when not to use**, and **examples/use-cases** in a clear table.

✅ NOTE: In real projects, companies often use **hybrid combinations** (example: "Hybrid Cloud + Blue/Green + Canary + Rolling").

✅ Types of Deployment (Real-Time) with Use Cases (Table)

#	Deployment Type	What it Means (Simple)	Best Use Cases (Real-Time)	Example (Industry)	When NOT to Use
1	**On-Premise Deployment**	App runs on company-owned servers/datacenter	Govt systems, banks, defense, highly regulated environments	Core banking system in private datacenter	When scalability + speed of delivery is required
2	**Cloud Deployment**	App runs on AWS/Azure/GCP	Startups, SaaS apps, scalable products	Netflix (AWS)	When data residency rules prohibit cloud
3	**Public Cloud Deployment**	Shared cloud infra (AWS/Azure/GCP)	Most web apps, mobile apps, analytics	E-commerce app on AWS	Ultra-sensitive workloads requiring full isolation
4	**Private Cloud Deployment**	Cloud-like infra but only for one organization	Enterprises, regulated industries	Hospital running OpenStack private cloud	Small teams due to high cost/maintenance
5	**Hybrid Cloud Deployment**	Mix of on-prem + cloud	Enterprises migrating gradually	Bank uses on-prem DB + cloud app layer	When complexity outweighs benefits
6	**Multi-Cloud Deployment**	Using >1 cloud providers	Avoid vendor lock-in, high availability	App on AWS + Azure DR	When team is small (ops becomes difficult)
7	**Bare Metal Deployment**	App runs directly on physical machines	High-performance compute, low latency	Gaming server infrastructure	When quick scaling is needed

8	**Virtual Machine (VM) Deployment**	Deploy app on VMs	Traditional enterprises, stable workloads	Java app on EC2 VM	When microservices agility is needed
9	**Container Deployment (Docker)**	Deploy app inside containers	Microservices, CI/CD, portability	Backend in Docker containers	Very small/simple apps where overhead is high
10	**Kubernetes Deployment (K8s)**	Container orchestration & scaling	Large microservices, autoscaling	E-commerce microservices on Kubernetes	Small apps—K8s complexity isn't worth it
11	**Serverless Deployment**	No server management (Function-based)	Event driven systems, automation	AWS Lambda file processing	Long-running tasks or heavy compute
12	**Edge Deployment**	Runs near the user/device for low latency	IoT, factories, smart cities	CCTV AI detection on edge device	Workloads that need centralized compute
13	**Mobile Deployment**	App deployed to mobile devices	Android/iOS apps	WhatsApp mobile app	Not applicable for backend APIs
14	**Desktop Deployment**	Software installed on PC/laptops	ERP client software, internal tools	Tally / SAP GUI	For internet-first apps
15	**Embedded Deployment**	Runs inside hardware device	IoT sensors, routers, smart gadgets	Firmware inside smart meter	Complex cloud apps
16	**Monolithic Deployment**	Entire app deployed as one unit	Small/medium apps, simple teams	Single Spring Boot WAR	Large teams with frequent changes
17	**Microservices Deployment**	Deploy each service separately	Big scalable systems	Uber microservices architecture	When team is small (too much infra overhead)
18	**Rolling Deployment**	Gradual replace old	Zero-downtime updates	Kubernetes RollingUpda	Risky for breaking

		pods/instances with new		te	changes
19	**Recreate Deployment**	Stop old version → start new version	Apps that can tolerate downtime	Internal admin tools	Production systems needing zero downtime
20	**Blue-Green Deployment**	2 environments (Blue old, Green new), switch traffic	Fast rollback deployments	Banking apps upgrades	When infra cost must be minimal
21	**Canary Deployment**	Release to small % of users first	High-risk releases, feature testing	Instagram feature rollout	When monitoring/metrics are weak
22	**A/B Testing Deployment**	2 versions for user groups for behavior testing	UI/UX experiments, marketing tests	Amazon checkout experiments	Backend logic critical systems (banking)
23	**Shadow Deployment**	Send real traffic to new version silently (no response)	Validate new system performance	Migrating search engine	When costs are high
24	**Feature Toggle Deployment**	Deploy code but enable features via flags	Continuous delivery	Feature flags in fintech apps	When flags become unmanageable
25	**Immutable Deployment**	New infra created fresh each release	Highly stable production, clean rollback	Auto-scaling groups with new AMI	When infra cost/time is limited
26	**Stateful Deployment**	Apps that store state locally (DB, Kafka, etc.)	DB, message brokers	PostgreSQL, Kafka cluster	Stateless services where it's unnecessary
27	**Stateless Deployment**	App doesn't store local state (horizontal scaling)	REST APIs, web servers	Spring Boot APIs	Apps needing local session/state

✅ Most Used Deployment Types in Real-Time (Top 10)

These are the most common in modern IT companies:

1. Cloud deployment
2. Docker container deployment
3. Kubernetes deployment
4. Rolling deployment
5. Blue-Green deployment
6. Canary deployment
7. Serverless deployment
8. Feature toggle deployment
9. Microservices deployment
10. Hybrid cloud deployment

If you want, I can also make a **separate table only for release deployment strategies** (Rolling, Canary, Blue/Green, etc.) with step-by-step workflow diagrams (Dev → QA → UAT → Prod).

Sure ✅ — below is a **real-time deployment decision table** (very practical), showing **which deployment to use** and **when to NOT use**.

✅ Which Deployment to Use vs When NOT to Use (Real-Time Table)

Deployment Type	✅ Use When (Best Fit)	❌ Do NOT Use When
On-Premise Deployment	Data is highly sensitive (banking, defense), strict compliance, internal network only	You need fast scalability, low infra maintenance, faster releases
Public Cloud Deployment (AWS/Azure/GCP)	Need quick deployment, scaling, global access, startup/SaaS products	Compliance requires full internal hosting, internet restrictions
Private Cloud Deployment	Enterprise needs isolation + cloud benefits, internal governance required	Team is small (maintenance expensive), workload is simple
Hybrid Cloud Deployment	You want gradual cloud migration, on-prem DB + cloud apps, DR planning	Architecture must stay simple, limited DevOps skills
Multi-Cloud Deployment	Avoid vendor lock-in, high availability across cloud providers	Budget/team limited, no strong monitoring/DevOps maturity

Bare Metal Deployment	Ultra-low latency + high performance (gaming, HPC, AI infra)	You need elasticity and quick scaling (bare metal scaling is slow)
Virtual Machine (VM) Deployment	Legacy apps, lift-and-shift migrations, stable workloads	You need microservices agility, faster auto scaling
Container Deployment (Docker)	Portability required, microservices, consistent environment	Small apps where Docker adds overhead unnecessarily
Kubernetes Deployment (K8s)	Many microservices, autoscaling, high availability, CI/CD	Small team, small app, no DevOps experience (complex ops)
Serverless Deployment (AWS Lambda / Azure Functions)	Event-driven apps, low maintenance, unpredictable traffic	Long-running tasks, heavy compute, low-latency constant workloads
Edge Deployment	IoT, factory automation, smart city, low-latency local processing	Apps needing centralized control/data storage
Monolithic Deployment	Small/medium projects, single team, quick initial product	Large teams, frequent releases, independent scaling required
Microservices Deployment	Big systems, independent scaling, multiple teams, fast releases	Small team/project, no automation, high operational complexity
Rolling Deployment	Need zero downtime gradual upgrades, Kubernetes standard approach	Breaking changes in DB/API not backward compatible
Recreate Deployment	App can tolerate downtime, batch systems, internal tools	Production services requiring high availability
Blue-Green Deployment	Need instant rollback, critical systems, clean release cutover	Infra budget limited (requires duplicate environment)
Canary Deployment	Risky changes, release gradually, validate with real users	No monitoring/logging, weak alerting, no SRE process
A/B Testing Deployment	UI/UX experiments, product optimization, marketing testing	Core business logic (finance, banking) where variation is dangerous
Shadow Deployment	Validating new system against real traffic before switching	High infra cost, sensitive user-data duplication risk

Feature Toggle Deployment	Continuous delivery, release without exposing features	Flags are not managed well → tech debt increases
Immutable Deployment	You want clean deployment, strong rollback (new infra each release)	Infra cost must be minimal, long provisioning time
Stateful Deployment	For DB, Kafka, caches requiring local storage	Stateless app services—stateful adds complexity unnecessarily
Stateless Deployment	REST APIs, scalable web apps, horizontal scaling	When the app requires session/state stored locally without shared store

---

✅ Quick Recommendation (Real Industry)

If you're building most modern projects:

✅ **Best default combo**

- **Cloud + Docker + Kubernetes**
- Release strategy: **Rolling** + (**Canary for critical features**)

If you want, I can make a **1-page decision matrix** like:

□ *Startup vs Enterprise vs Banking vs Government*
and map **exact deployment choice** for each.

Below is a **complete deployment-model mapping table** that shows **which type of software is suitable for which deployment model**, with real-world examples.

✅ Deployment Models vs Suitable Software (Real-Time Mapping Table)

Deployment Model	Suitable Software Type (Best Fit)	Real-Time Use Cases	Examples
On-Premise (On-Prem)	Banking systems, government apps, internal ERP, defense software	Data cannot leave organization, strict compliance	Core Banking, Govt portals, Defense apps

Public Cloud	SaaS apps, e-commerce, OTT platforms, scalable web apps	Need fast scaling + global reach	Netflix, SaaS CRM, shopping apps
Private Cloud	Enterprise internal platforms, healthcare systems, regulated industries	Cloud benefits but isolated infra required	Hospital systems, Insurance systems
Hybrid Cloud	Enterprises migrating to cloud, mixed workloads	Some parts on-prem, some on cloud	Bank: on-prem DB + cloud app
Multi-Cloud	Mission-critical apps, DR-heavy systems	Avoid vendor lock-in, strong availability	Global fintech app with AWS+Azure
Bare Metal	HPC systems, AI training infra, gaming hosting	Ultra-high performance required	AI GPU servers, Gaming matchmaking infra
Virtual Machine (VM)	Legacy apps, monolithic enterprise software, stable systems	Lift-and-shift workloads	Java legacy ERP on EC2 VM
Container (Docker)	Microservices apps, CI/CD-friendly apps	Fast delivery, consistent runtime	API services, backend apps
Kubernetes (K8s)	Large microservices systems, enterprise SaaS	Auto scaling, HA, service orchestration	Uber-like systems, microservice platforms
Serverless (FaaS)	Event-driven systems, automation tools	Pay-per-use, low operations	Email triggers, file processing pipelines
PaaS Deployment	Web apps, internal dashboards, APIs	Faster development, less infra	Heroku apps, Azure App Service
SaaS Deployment	Business apps sold to customers	Subscription-based products	Google Workspace, Salesforce
Desktop Deployment	Offline tools, enterprise client software	Needs local machine processing	Tally, SAP GUI, Accounting apps
Mobile Deployment	Consumer apps, fintech apps, social apps	Runs on Android/iOS	WhatsApp, Paytm, Instagram
Embedded Deployment	IoT + hardware software	Runs in sensors/devices	Smart meters, routers, IoT devices

Edge Deployment	Low-latency IoT + real-time analytics	Process near device	CCTV face detection on edge
CDN Deployment	Media apps, static websites, streaming platforms	Fast content delivery	Static website + CDN
Monolithic Deployment	Small/medium apps, early-stage startups	Simple structure	Single Spring Boot project
Microservices Deployment	Large enterprise apps, big teams	Independent scaling + deployments	Amazon-like commerce platform
SOA Deployment	Enterprise service integration	Heavy enterprise middleware	Telecom CRM integration
Client-Server Deployment	Traditional enterprise apps	Centralized server with clients	Office internal apps
Distributed Deployment	Large-scale systems, global users	Reliability + replication	Distributed databases
Peer-to-Peer Deployment	Decentralized apps	Users share resources	Torrent systems
Stateful Deployment	Databases, messaging systems	Requires storage persistence	PostgreSQL, Kafka clusters
Stateless Deployment	APIs, web apps	Best scalability	REST APIs, frontend apps
Rolling Deployment	APIs/microservices	Zero downtime upgrades	Kubernetes RollingUpdate
Blue-Green Deployment	Critical enterprise systems	Instant rollback	Banking transaction system updates
Canary Deployment	Feature releases, risky updates	Release gradually	Instagram feature rollout
Recreate Deployment	Internal tools, low-traffic apps	Downtime is OK	Admin dashboards
A/B Testing Deployment	UI/UX products	Experimentation	E-commerce checkout testing
Shadow Deployment	Migration/testing	Real traffic testing silently	Search engine replacement validation

Feature Toggle Deployment	Continuous delivery apps	Deploy anytime without enabling	Fintech feature releases with flags
Immutable Deployment	High reliability systems	Clean rollback / new infra each time	AMI-based deployments
GitOps Deployment	Kubernetes-based enterprise systems	Version-controlled infra deployment	ArgoCD/Flux-based systems
CI/CD Pipeline Deployment	All modern apps	Automated builds & releases	Jenkins/GitHub Actions releases

---

✅ **Quick Guide (Most Common Mapping)**

Software Category	Best Deployment
Small project (single team)	Monolith + VM / PaaS
Startup product	Public Cloud + Docker
Enterprise product	Hybrid cloud + Kubernetes
Fintech / Banking	On-prem / Private cloud + Blue-Green
IoT + CCTV	Edge deployment
Automation workflows	Serverless
Big scalable apps	Microservices + K8s + Canary

If you want, I can also provide a **"Deployment Model Selection Chart"** (decision matrix) like:

✅ **App Size (small/medium/large)** + ✅**Compliance** + ✅**Traffic pattern**
→ tells exactly which deployment to choose.